XV6 Lab Assignment: TLB and Page Fault Measurement

Objective:
Understand TLB behavior, page faults, and lazy allocation in XV6 by implementing and running `tlbrun` and `tlbtest`.

Part 1: Preparing the Syscall

1. Add a `page_faults` field in the `proc` struct
   File: `kernel/proc.h`

```
struct proc {
    …
    int page_faults;   // count of page faults
    …..
};
```

2. Initialize `page_faults` in `allocproc()`
   File: `kernel/proc.c`

```
// Inside allocproc() function
……
// Initialize page fault counter
p->__________ = ________;  //your code
```

File: kernel/proc.c in growproc() make changes

```
 if(n > 0){
    sz += n;   // just bump size, do not allocate
  } else if(n < 0){
    sz = uvmdealloc(p->pagetable, sz, sz + n);}
```

3. Update syscall.h

```
#define SYS_getpagefaults 24   // choose a free number
```

4. Update usys.S

```
.global getpagefaults
getpagefaults:
 li a7, SYS_getpagefaults
 ecall
 ret
```

5. Add a syscall to get page faults

File: kernel/sysproc.c

```
uint64 sys_getpagefaults(void) {
    struct proc *p = myproc();
    return p->page_faults;
}
```

6. Add declaration in user.h

```
int getpagefaults(void);
```

7. Add this line to usys.pl

```
entry("getpagefaults");
```

8. Add this lines to syscall.c

```
extern uint64 _____________ (void); //complete this
```

```
[SYS_getpagefaults] = _________ ,  //complete this
```

Part 2: Modifying VM for Lazy Allocation

1. Update vmfault declaration
   File: kernel/defs.h

```
int      vmfault(pagetable_t, uint64, int);
```

2. Implement vmfault

File: kernel/vm.c update the existing vmfault() function as mentioned below:

```
int

vmfault(pagetable_t pagetable, uint64 va, int write)

{

  struct proc *p = myproc();

  void *mem;


  if (va >= p->sz)

    return -1;
```

```
  va = PGROUNDDOWN(va);

  // already mapped? then nothing to do
  if (walkaddr(pagetable, va) != 0)
    return 0;

  mem = kalloc();
  if(mem == 0)
    return -1;

  memset(mem, 0, PGSIZE);

  if (mappages(p->pagetable, va, PGSIZE, (uint64)mem,
               PTE_R | PTE_W | PTE_X | PTE_U) != 0) {
    kfree(mem);
    return -1;
  }
return 0;}
```

4. Add this line to vm.h

```
int vmfault(pagetable_t pagetable, uint64 va, int write);
```

Part 3: Modifying usertrap for Page Fault Counting

1. File: kernel/trap.c
   In usertrap(), for load/store page faults:

```
    else if (r_scause() == 15 || r_scause() == 13) {
          p->page_faults++;
 if (vmfault(p->pagetable, r_stval(), (r_scause() == 13) ? 1 : 0) != 0)
{
            // vmfault failed: kill the process
            setkilled(p);
    }
    }
```

PART 4:Make tlbrun.c and tlbtest.c in user file and Complete the below code

1.tlbrun.c

```
#include "kernel/types.h"
#include "user/user.h"
#define PAGESIZE 4096
#define MAXPAGES 1024

int main(int argc, char *argv[]) {
    int jump = PAGESIZE / sizeof(int);
     int i,t;

    printf("PageCount\tTrials\tTicks\tPageFaults\n");

    for (int numpages = 1; numpages <= MAXPAGES; numpages *= 2)
{
        int trials = 5000000;
        int faults_before = getpagefaults();  // add this
```

```
        uint start = uptime();


       int *arr = (int *) sbrk(numpages * PAGESIZE);
        if (arr == (void *) -1) {
            printf("sbrk failed for %d pages\n", numpages);
            exit(1);
        }


        for (int t = 0; t < trials; t++) {
            for (int i = 0; i < (numpages/2) * jump ; i += jump)
{
                // Complete the touch pages
            }
        }
        uint end = uptime();
        int faults_after = getpagefaults();
   printf("%d\t\t%d\t%d\t%d\n",numpages, trials, end - start,
faults_after - faults_before);
    }
exit(0);
}
```

2.tlbtest.c

```
// user/tlbtest.c
#include "kernel/types.h"
#include "user/user.h"

#define PAGESIZE 4096
#define MAXPAGES 1024

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: tlbtest <numpages> <trials>\n");
        exit(1);
    }

    int numpages = atoi(argv[1]);
    int trials = atoi(argv[2]);

    if (numpages < 1 || numpages > MAXPAGES) {
```

```
    printf("numpages out of range (1..%d)\n", MAXPAGES);
    exit(1);
}

int jump = PAGESIZE / sizeof(int);
int i, t;

// Dynamically allocate pages using sbrk()
int *arr = (int *) sbrk(numpages * PAGESIZE);
if (arr == (void*) -1) {
    printf("sbrk failed for %d pages\n", numpages);
    exit(1);
}

// Get page fault count before test
int pf_before = getpagefaults();

// Start timing
uint start_ticks = uptime();

// Access pages to trigger lazy allocation page faults
```

```
    for (t = 0; t < trials; t++) {
        for (i = 0; i < (numpages/2) * jump; i += jump) {
            arr[i] += 1;   // first access triggers page fault
        }
    }

    // End timing
    uint end_ticks = uptime();

    //Write code for page fault count after test

    printf("Pages: %d\tTrials: %d\tTicks: %d\tPageFaults: %d\n",
           numpages, trials, end_ticks - start_ticks, pf_after -
pf_before);

    exit(0);
}
```

3.Add to UPROGS in Makefile:

```
UPROGS=\
    _tlbrun\
    _tlbtest
```

4.Build and run xv6:

```
make clean
make
make qemu
```

5.$ tlbrun

```
PageCount    Trials    Ticks    PageFaults
1        5000000         0    0
2        5000000  `      1    1
4        5000000         0    2
8        5000000         1    4
. . .
1024        5000000      112    512
```

6. $tlbtest <numpages> <trials>

$ tlbtest 16 5000000

Pages: 16   Trials: 5000000   Ticks: 2   PageFaults: 8